Encapsulation and Access Methods: Getters and Setters

Research Assignment

Object-Oriented Programming (SEP–DEC 26)

Encapsulation and Information Hiding

Zavala Flores Ángel

Eng. Pedro Said Vara Chacón

October 7, 2026

1. Introduction

Every object-oriented program manages *state*: the values an object retains between one operation and the next. When that state can be modified by any part of the system, every line of code becomes a potential source of errors, and reasoning about the correctness of the program requires knowing all the code that ever touches that data. **Encapsulation** addresses this problem by grouping data and behavior into a single unit (the class) and restricting direct access to the internal state.

This principle is related to the **information hiding** proposed by Parnas (1972): each module should hide its design decisions that are most likely to change behind a stable interface. Accessor methods (*getters*) and modifier methods (*setters*) are the classic mechanism by which a class decides which part of its state is exposed and under what rules it may change.

This report analyzes the foundations of encapsulation, the risks of direct access, the role of mutators as guardians of business invariants, read-only and write-only properties, and the debate over the "boilerplate antipattern" (*boilerplate*). It is complemented by a comparative case in Python (the `BankAccount` class) whose code was executed to obtain the evidence included, and by an individual reflection on the *Tell, Don't Ask* principle.

2. Core Analysis

Principle of Encapsulation and Information Hiding

Encapsulation has two facets: *grouping* related data and operations, and *restricting* access to internal details. Information hiding is the underlying reason for the second: the class publishes a *contract* (its interface) and keeps its *implementation* private. As long as the contract is respected, the implementation can change (for example, storing the balance as whole cents instead of decimals) without affecting client code.

This is why attributes should be **private or protected by default**:

- **Protection of invariants:** only the class can guarantee that its state is always valid (e.g., a non-negative balance).
- **Low coupling:** client code depends on operations, not on the internal representation, so the latter can evolve freely.
- **Localization of changes and errors:** if the state becomes corrupted, the culprit is inside the class and not spread across the whole system.
- **Smaller API surface:** whatever is published becomes a commitment; whatever is private can be modified without breaking anyone (Bloch, 2018).

Bloch (2018, Item 15) summarizes the practical rule: *minimize the accessibility of classes and members*. Expose only what is strictly necessary, and widen access only when there is a justified reason. In languages such as Java there are the `private` and `protected` modifiers; in Python, hiding relies on convention (the `_` prefix) and on the *name mangling* of the double `__` prefix, together with the `@property` mechanism (Python Software Foundation, 2024).

2.2 Technical Definitions

Accessor (*getter*). A method that returns the value (or a derived value) of an object's attribute. Usual signature: `ReturnType getAttribute()` in Java (or `isAttribute()` for booleans); in Python, a function decorated with `@property`. It takes no parameters and **must not have observable side effects**: calling it several times in a row must return the same result if the state has not changed (Meyer's command–query separation principle, 1997). When the attribute is mutable, a good accessor returns a *defensive copy* or an immutable view so as not to leak the internal reference.

Mutator (*setter*). A method that modifies the internal state of the object. Usual signature: `void setAttribute(Type value)`. Its **main responsibility** is to act as the single, controlled point of change: validate the input, reject or normalize inadmissible values and, where appropriate, maintain consistency with other dependent attributes, so that the object never moves into an invalid state.

Part II. Architectural Analysis

When attributes are public and any client code can assign them (for example, `account.balance = -500`), several design problems appear:

- **Invalid states:** nothing prevents negative balances, empty holders, or incorrect types. The business rules are left without an owner.
- **Scattered and duplicated validation:** every client must remember to validate; a single oversight is enough to corrupt the object. The rules are copied in multiple places and diverge.
- **Coupling to the representation:** if the internal structure is later changed (e.g., from *float* to *Decimal* or cents), all the code that assigned the field directly breaks.
- **Inability to add behavior:** auditing cannot be logged, observers cannot be notified, and security cannot be applied on each change.
- **Difficult debugging:** the wrong value could have originated anywhere; there is no single place to put a *breakpoint* or an assertion.
- **Concurrency problems:** without a control point, there is no natural place to synchronize access to shared state.

2.4 Enforcement of Business Invariants

An *invariant* is a condition that must always hold whenever the object is observable from the outside (Meyer, 1997). Mutators (and constructors) are the **guardians** that protect it: they receive a change request, check that the resulting state would be valid, and only then apply it; otherwise they reject the operation by throwing an exception or returning an explicit error. The general pattern is:

```
def set_x(self, new_value):
    if not is_valid(new_value):      # 1. validate
        raise ValueError(...)        # 2. reject (fail fast)
    self.__x = normalize(new_value)  # 3. change only if safe
```

By centralizing the rule in a single place, changing it (e.g., allowing an overdraft up to a certain limit) requires modifying only the class, not its clients. In addition, validating in the constructor guarantees that the object is *born* in a valid state, avoiding "half-built" objects.

2.5 Read-Only and Write-Only Properties

Selectively implementing accessors and mutators defines access boundaries:

Configuration	Effect	Example
Getter only	Read-only property; the value changes only through internal operations.	*balance*: modified with `deposit()` / `withdraw()`, never by direct assignment.
Setter only	Write-only property; it can be set	*password*: it is assigned, its hash

Configuration	Effect	Example
	but not read.	is stored, and it is never returned.
Getter + setter	Controlled reading and writing, with validation.	*target temperature* of a thermostat within a range.
None	Implementation detail, completely hidden.	Internal cache counter.

In Python, omitting the `@x.setter` turns the property into a read-only one: trying to assign it raises `AttributeError`, as will be seen in Case B.

2.6 The "Boilerplate Antipattern" (*boilerplate*)

Question: does generating getters and setters for every private field defeat the purpose of encapsulation? **Answer: yes, when done blindly and mechanically; no, when designed with intent.** Justification:

- A `getX/setX` pair with no validation over a private field is, functionally, a public field with more text. The form is satisfied (the attribute is private) but the substance is lost (hiding the representation and protecting invariants). The client can still leave the object in any state and remains coupled to its internal structure.
- Trivial accessors expose the *representation*. Martin (2009, ch. 6) warns that abstraction does not consist of adding layers of getters and setters over variables, but of exposing abstractions that allow the user to manipulate the *essence* of the data without knowing its implementation.
- Public setters make objects mutable that could be immutable, and Bloch (2018, Item 17) recommends minimizing mutability.
- Fowler (2006) calls the effort to eliminate getters by moving logic to where the data lives the *getter eradicator*; each getter is a sign that some logic might belong in the class and not in its client.

Nuance: accessors are not bad *per se*. They are legitimate for *data transfer objects* (DTOs), for queries that are part of the contract (`account.balance`) and as a future extension point. The antipattern is **indiscriminate automatic generation**: each accessor must be justified by a domain need, not by the existence of a field.

Part III. Comparative Case Study

The `BankAccount` class is used in Python 3. Both fragments were executed and the outputs shown are real.

Case A: Flawed Design (Public Attributes)

```
class BankAccount:
    """Case A: flawed design (public attributes)."""
    def __init__(self, holder, balance):
        self.holder = holder
```

```
        self.balance = balance

account = BankAccount("Ana", 1000)
account.balance = -500              # invalid state accepted silently
account.holder = ""                 # empty holder
account.balance = "lots of money"   # wrong type
print(account.balance, repr(account.holder))
```

Execution output:

```
lots of money ''
```

The external code left the account with a text-type balance and an empty holder. No business rule was consulted. (Assigning `-500` as the balance would also have been accepted.) The class cannot guarantee anything about itself.

Case B: Refactored Design (Private State, Accessors, and Validation)

```
class BankAccount:
    """Case B: refactored design (private state, validation)."""
    def __init__(self, holder, initial_balance=0.0):
        if not holder or not holder.strip():
            raise ValueError("The holder cannot be empty.")
        self.__holder = holder.strip()
        self.__balance = 0.0
        if initial_balance:
            self.deposit(initial_balance)

    # Accessors (getters): read-only, no side effects
    @property
    def holder(self):
        return self.__holder

    @property
    def balance(self):
        return self.__balance

    # Mutator with validation (gatekeeper)
    @holder.setter
    def holder(self, new_value):
        if not new_value or not new_value.strip():
            raise ValueError("The holder cannot be empty.")
        self.__holder = new_value.strip()

    # Behavior-oriented methods (Tell, Don't Ask)
    def deposit(self, amount):
        if not isinstance(amount, (int, float)) or amount <= 0:
            raise ValueError("Deposit must be a positive number.")
        self.__balance += amount

    def withdraw(self, amount):
```

```
        if not isinstance(amount, (int, float)) or amount <= 0:
            raise ValueError("Withdrawal must be a positive number.")
        if amount > self.__balance:
            raise ValueError("Insufficient funds.")
        self.__balance -= amount

c = BankAccount("Ana", 1000)
for attempt in (lambda: c.deposit(-500), lambda: c.withdraw(5000),
                lambda: setattr(c, "balance", -500),
                lambda: setattr(c, "holder", "")):
    try:
        attempt()
    except (ValueError, AttributeError) as e:
        print(type(e).__name__ + ":", e)
c.withdraw(300)
print("Final balance:", c.balance)
```

Execution output:

```
ValueError: Deposit must be a positive number.
ValueError: Insufficient funds.
AttributeError: property 'balance' of 'BankAccount' object has no sette
r
ValueError: The holder cannot be empty.
Final balance: 700.0
```

Observations on the refactored design:

- **Private state** (`__balance`, `__holder`): direct external access is no longer possible.
- **Accessors without side effects:** `balance` and `holder` only return values.
- **Read-only balance:** it has no setter; assigning it produces `AttributeError`. It changes only through `deposit()` and `withdraw()`.
- **Explicit validation in a mutator:** the setter for `holder` rejects empty strings; `withdraw()` rejects non-positive amounts and withdrawals that exceed the balance (invariant: balance >= 0).
- **Invariant from the constructor:** an account with an empty holder cannot be created.

Note that the final class has almost no setters: most of the control is expressed through behavior methods, which connects with the reflection in the next section.

3. Individual Reflection

When are accessors and mutators really needed? I consider them necessary in three situations: (1) when the data is a natural part of the object's contract and clients legitimately need to query it (the balance, the holder's name); (2) when the object is primarily a *data carrier* (DTOs, persistence entities, messages), where the logic does not live in the class; and (3) when a framework requires it (serialization, interface *binding*, ORM). A mutator, in particular, is only justified if directly changing that attribute is a meaningful operation for the domain and if it can be validated independently of the rest of the state.

The "Tell, Don't Ask" principle. This principle, described by Fowler (2013) based on guidance from Hunt and Thomas, says that instead of *asking* an object for its state and making decisions outside of it, you should *tell* it what to do and let it decide internally. Compare:

```
# Ask (the logic lives in the client and duplicates the rule):
if account.balance >= amount:
    account.balance = account.balance - amount
else:
    show_error()

# Tell (the logic lives where the data is):
account.withdraw(amount)   # the account decides whether to proceed and
 raises an error if not
```

The "Ask" version has a logical race condition (the balance may have changed between the read and the write), forces every client to replicate the rule, and requires a public setter for *balance* that destroys the invariant. The "Tell" version encapsulates the decision: the object is responsible for its own consistency, and the contract expresses domain intent (*withdraw*) instead of data mechanics (*assign balance*).

Personal conclusion. The starting point should be to ask *what behavior the client needs*, not *what fields the class has*. I would first design methods with domain names (`deposit`, `withdraw`, `transfer`), add read-only accessors only for the data that clients really need to query, and resort to a setter only when there is an explicit reason and a validation to back it up. In this way, encapsulation stops being a syntactic formality (private fields) and becomes a real guarantee: the object protects its own state.

4. References

Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.

Fowler, M. (2006, December 7). *GetterEradicator*. martinfowler.com. https://martinfowler.com/bliki/GetterEradicator.html

Fowler, M. (2013, March 22). *TellDontAsk*. martinfowler.com. https://martinfowler.com/bliki/TellDontAsk.html

Martin, R. C. (2009). *Clean Code: A handbook of agile software craftsmanship*. Prentice Hall.

Meyer, B. (1997). *Object-oriented software construction* (2nd ed.). Prentice Hall.

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. *Communications of the ACM, 15*(12), 1053-1058. https://doi.org/10.1145/361598.361623

Python Software Foundation. (2024). *Built-in functions: property*. Python 3 documentation. https://docs.python.org/3/library/functions.html#property